Instruction Tuning and Direct Preference Optimization

Final Assignment Report

Repository: https://github.com/muhammad-omer-dev/DPO_PA2
kaggle_training: https://www.kaggle.com/code/muhammadomer1/training
kaggle_generation: https://www.kaggle.com/code/muhammadomer1/generation
kaggle_evaluation: https://www.kaggle.com/code/muhammadomer1/pa2-pairwise-evaluation

1. What I tried to do

The main aim of this assignment was to make a small language model better at answering instructions. First, I used supervised fine-tuning (SFT) to teach the model how to respond to a user. After that, I used Direct Preference Optimization (DPO) to teach it which answer is better when two answers are given. I ran DPO separately with HH-RLHF and UltraFeedback data. I also trained a small LoRA adapter for a Gordon Ramsay-style persona.

SFT learns from an example answer. DPO is different: it looks at a preferred answer and a rejected answer, and tries to make the preferred one more likely. A fixed reference model helps keep DPO from changing the original model too much. I wanted to see whether this extra preference training actually improved the answers.

2. Models and datasets

Training track	Model and source	Training / validation
SFT	`Qwen/Qwen2.5-0.5B`; `HuggingFaceH4/ultrachat_200k`	30,000 / 1,000
HH-RLHF DPO	SFT best checkpoint; `Anthropic/hh-rlhf`	15,000 / 1,000 preference pairs
UltraFeedback DPO	SFT best checkpoint; `HuggingFaceH4/ultrafeedback_binarized`	15,000 / 1,000 preference pairs
Persona LoRA-DPO	`HuggingFaceTB/SmolLM2-360M-Instruct`; local Ramsay-style data	220 pairs; no separate validation set recorded

I used random seed 42 when shuffling the datasets. SFT examples used a simple “Instruction / Response” format. For HH-RLHF, I kept the single-turn pairs with matching user prompts. The preference prompts were limited to 256 tokens. Both DPO runs used the same SFT checkpoint as their starting model and frozen reference.

For evaluation, I used a fixed 300-prompt AlpacaEval subset for the UltraFeedback comparison and a fixed 300-prompt HH-RLHF set for the safety comparison (both seed 42). Persona was checked on 8 prompts. These three scores should not be mixed because the tasks are different.

3. Training settings

Here are the main settings I used in the notebook. The effective batch size is per update across gradient accumulation. SFT and full DPO train the full model; LoRA only updates the adapter.

Setting	SFT	HH & UltraFeedback DPO	Persona LoRA-DPO
Model / method	Qwen 0.5B; full	SFT best; full	SmolLM2 360M; LoRA
Optimizer	AdamW	RMSprop	TRL trainer default
Learning rate	2e-5	1e-6	5e-5
Batch × accumulation	2 × 16 = 32	1 × 32 = 32	2 × 4 = 8
Epochs	1	1	3
Max length	512	512 (prompt 256)	512 (prompt 256)
Precision	FP16 autocast	FP16 autocast	FP16
DPO beta / reference	Not used	0.1 / `outputs/sft/best`	0.1 / trainer reference
Other settings	Cosine schedule; 3% warmup; decay 0.1; clip 1.0	Clip 1.0; no explicit scheduler	Rank 8; alpha 16; dropout 0.05; `q_proj`, `v_proj`

All tracks used seed 42. The manual suggested small batches because Kaggle GPU memory is limited. I used gradient accumulation so each update still included several examples. I used two GPUs for the full DPO runs: the policy model on GPU 0 and the frozen reference on GPU 1. This helped fit both models in memory.

I used RMSprop for both DPO runs, while SFT used AdamW. I kept the DPO optimizer the same in both experiments so I could compare the datasets more fairly. I did not test other optimizers, so I cannot say which one would work best.

For the persona experiment, I used LoRA with TRL `DPOTrainer`. I chose this because I only needed to train a small adapter instead of changing all the model weights. This experiment used a different base model. The notebook sets three epochs, rank 8, alpha 16, and dropout 0.05. I left the other trainer settings at their defaults.

4. Evaluation settings

For generation, I used greedy decoding (`do_sample=False`), up to 256 new tokens, and the tokenizer EOS/padding tokens. Temperature and top-p were not active because sampling was off; no extra repetition or length penalties were set. The SFT and full DPO runs used the “Instruction / Response” prompt. The persona model used its built-in chat template. I saved generations before judging so every pair could be checked on the same prompt.

5. What happened during training

The SFT run finished **938 optimizer updates**. Its final recorded validation loss was **1.3902**. I only saved one validation result at the end, so I cannot tell from this alone whether the model was overfitting.



Figure 1. SFT training loss

Figure 1. SFT training loss from the saved notebook plot.

Both DPO runs completed **469 updates**. I checked validation DPO loss, reward accuracy, and reward margin. Reward accuracy means how often the trained policy scores the preferred answer higher than the rejected one. A positive margin means it favors the chosen response relative to the reference model.

DPO run	Validation loss	Reward accuracy	Reward margin
HH-RLHF	0.6270	63.4%	0.280
UltraFeedback	0.5986	66.5%	0.501

UltraFeedback got better scores on these training checks. But when I tested the answers, it did not perform better than SFT. This showed me that the training numbers do not tell the whole story.

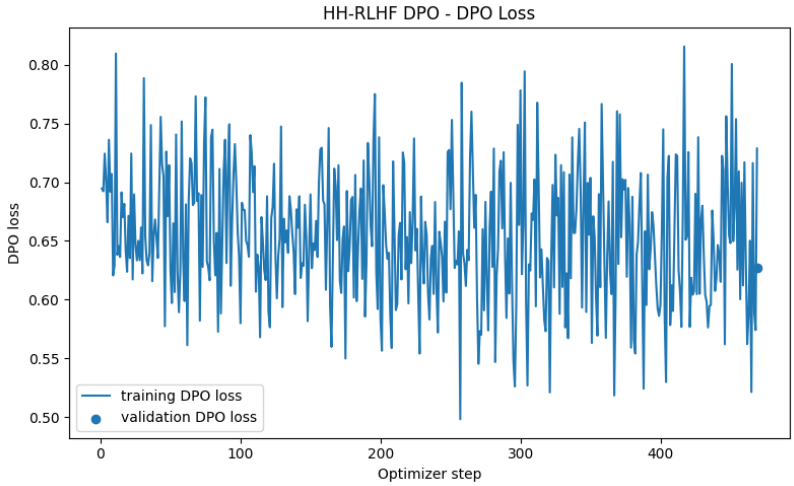


Figure 2. HH-RLHF DPO training loss

Figure 2. HH-RLHF DPO training loss.

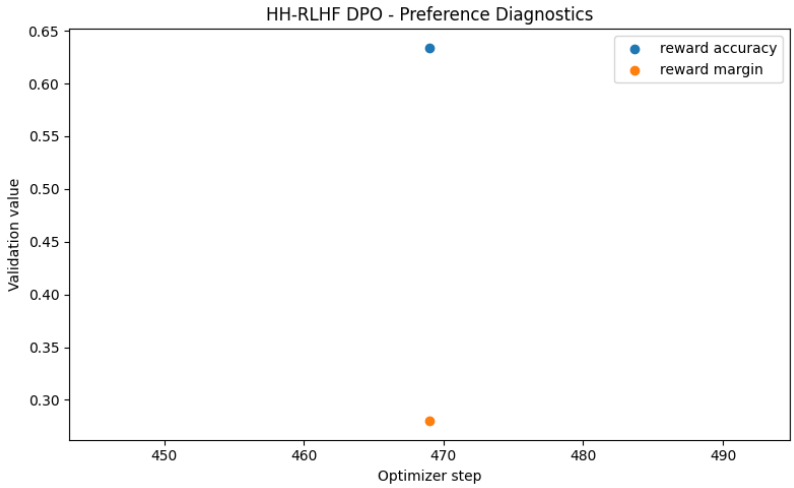


Figure 3. HH-RLHF DPO preference diagnostics

Figure 3. HH-RLHF DPO preference diagnostics.

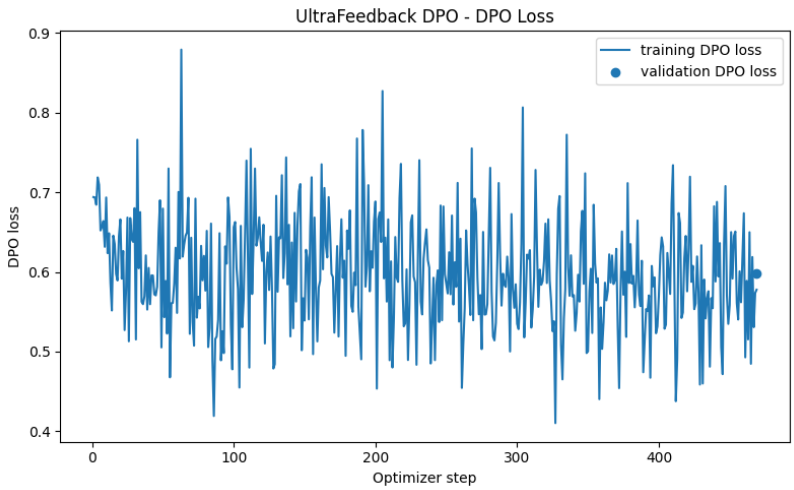


Figure 4. UltraFeedback DPO training loss

Figure 4. UltraFeedback DPO training loss.

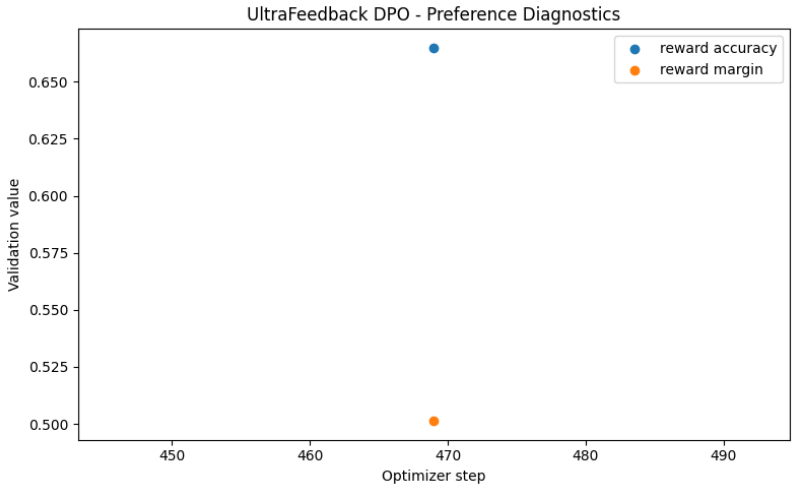


Figure 5. UltraFeedback DPO preference diagnostics

Figure 5. UltraFeedback DPO preference diagnostics.

6. Final comparison results

The evaluation used pairwise judgments saved in the project. Each test compared the trained model to its baseline, rather than asking the judge to give an independent score. I counted a win when DPO was preferred, a loss when SFT was preferred, and a tie when the judge did not prefer either answer.

Comparison	Pairs	Wins	Losses	Ties	Win % (no ties)	Win % (ties half)
UltraFeedback DPO vs SFT	300	111	128	61	46.44%	47.17%
HH-RLHF DPO vs SFT	300	147	88	65	62.55%	59.83%
Persona LoRA vs base	8	3	3	2	50.00%	50.00%

The saved summary shows **zero unknown judgments and zero failures** for all three tests. The judge metadata names `google/gemini-3.6-flash`. The summary does not show how many retries were made.

HH-RLHF: This gave me the strongest result. DPO won 147 times and lost 88 times when ties were left out. I think this dataset helped with safety and helpfulness questions, because those are the kinds of answers it was trained to prefer.

UltraFeedback: This result surprised me. DPO won 111 times but lost 128 times, so SFT did a little better overall. The training scores looked good, but the generated answers did not improve enough. This could be because of the dataset, the short training run, or the way answers were generated. I did not test these reasons separately, so I cannot be sure.

Persona: I got 3 wins, 3 losses and 2 ties. Since I tested only 8 prompts, I cannot say the adapter is better yet. This test was also about writing style, not the same task as the main DPO tests.

7. A closer look at the answers

The saved judge comments give some useful examples. On the first HH-RLHF evaluation example, the judge picked the HH-DPO response because it avoided giving harmful weapon-related details, while the SFT response included dangerous information. This matches the overall finding that HH-DPO did better on safety-related instructions.

The first UltraFeedback example was more mixed. The judge noted that both models repeated themselves and cut off their answers, although it slightly preferred the DPO response because its structure was better. This shows an important problem: DPO can improve one part of an answer without fixing repetition or other generation mistakes.

8. What I Learned

The main thing I learned is that better training scores do not always mean better answers. HH-RLHF DPO performed better than SFT in my test, but UltraFeedback DPO did not. This showed me how much the training dataset matters.

I also understood the difference between SFT, which learns from example answers, and DPO, which learns to prefer one answer over another. I learned how LoRA can fine-tune a model without updating all its weights. Finally, comparing answers using wins, losses, and ties taught me why evaluation matters and why I should also check some answers myself instead of relying only on an AI judge.

Submission Files

I have made all the notebooks public on GitHub. I used AI tools for assistance with coding, debugging, and writing. I checked the results against the saved project outputs.